

CHAPTER 7 — BINARY SEARCH ALGORITHM

Learning Goal

Understand the **step-by-step process of Binary Search**, how **low**, **high**, and **mid** control the search area, and how the search becomes smaller after every comparison.

SLIDE 1 — From Binary Search Idea to Algorithm

How do we make Binary Search work step by step?

We already know the main idea:

Check the middle → Compare → Choose left or right → Repeat

Now we turn this idea into an **algorithm**.

Example

Sorted array:

10 20 30 40 50 60 70

 Target = 60

First, we need to know the **search area**.

 **low** → starting position

 **high** → ending position

 **mid** → middle position

low
↓

high
↓

[10] [20] [30] [40] [50] [60] [70]

↑
mid

Memory Trick

LOW = Start

HIGH = End

MID = Middle

SLIDE 2 — The Binary Search Steps

Follow These Steps

1 Set the search area

low = first index

high = last index

2 Find the middle

$\text{mid} = (\text{low} + \text{high}) / 2$

3 Compare

 **target** with `numbers[mid]`

4 Decide

If:

`target == numbers[mid]`

 **Found!**

If:

`target < numbers[mid]`

 Search the **left half**

If:

`target > numbers[mid]`

➡ Search the **right half**

5 Repeat ↺

Continue until the target is found or the search area becomes empty.

🧠 **Memory Trick**

Start → Middle → Compare → Choose → Repeat

🟪 SLIDE 3 — How Does the Search Area Change?

✂️ **The Secret of Binary Search**

The search area keeps becoming smaller.

Example:

10	20	30	40	50	60	70
↑			↑	↑		
low			mid	high		

🎯 Target = 20

Since:

20 < 40

👉 We only need the **left half**.

10 20 30 | 40 50 60 70

🔍 Search | ❌ Ignore

Now:

low → 10

high → 30

The middle is recalculated.

 **Every round:**

Search Area → Middle → Compare → Smaller Search Area

 **Remember**

Binary Search doesn't randomly remove values.
It removes the half that **cannot contain the target**.

SLIDE 4 — Dry Run: Binary Search Step by Step

 **Let's Trace It!**

Array:

10 20 30 40 50 60 70

 Target = 60

Step	Low	High	Middle	Middle Value	Decision
1	0	6	3	40	60 > 40 → Right
2	4	6	5	60	60 = 60 → Found 

 **Visual**

Step 1

10 20 30 [40] 50 60 70

 Target is greater → Move right



Step 2

50 [60] 70

👉 Match! ✅

💡 **Memory Trick**

Compare → Eliminate → Recalculate

🟡 SLIDE 5 — Binary Search Algorithm in One Flow

🧠 **Put Everything Together**

START



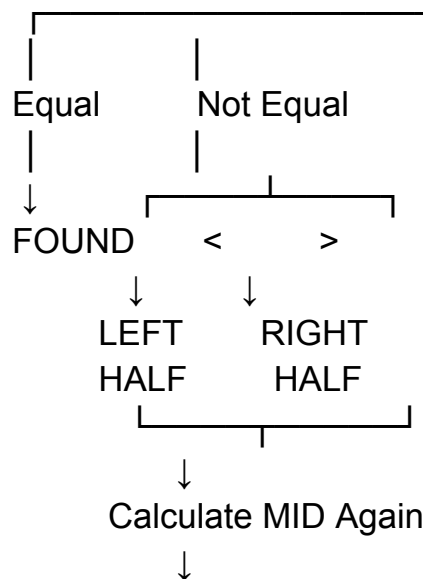
Set LOW and HIGH



Find MID



Compare TARGET with MID



REPEAT

Algorithm in Simple Words

- 1 Start with the first and last positions.
- 2 Find the middle position.
- 3 Compare the target with the middle value.
- 4 If equal → target found.
- 5 If target is smaller → move to left half.
- 6 If target is larger → move to right half.
- 7 Repeat until found or no search area remains.

BIG MEMORY TRICK

LOW → MID → COMPARE → LEFT/RIGHT → REPEAT

CHAPTER 7 — QUIZ

Q1 — Fill the Code

Complete the code to calculate the middle position:

```
int low = 0;  
int high = 6;
```

```
int mid = _____;
```

 **Hint:** The middle is calculated using the starting and ending positions.

Glossary

-  **low** — Starting index of the search area.
-  **high** — Ending index of the search area.
-  **mid** — Middle index of the search area.

✓ **Answer:**

$(\text{low} + \text{high}) / 2$

🟢 Q2 — Fill the Code

Complete the condition:

```
if(target _____ numbers[mid])
{
    printf("Found");
}
```

💡 **Hint:** This condition checks whether the target and middle value are equal.

📖 Glossary

- 🎯 **target** — The value we want to find.
- 🟡 **mid** — The middle position.
- 📦 **numbers[mid]** — The value stored at the middle position.

✓ **Answer:** ==

🟡 Q3 — MCQ

A sorted array is:

10 20 30 40 50 60 70

🎯 Target = 20

The middle value is 40.

What should Binary Search do next?

- A)** Search the right half
- B)** Search the left half

- C) Start from the beginning
- D) Stop immediately

💡 **Hint:** Compare 20 and 40.

📖 Glossary

- 🎯 **Target** — The value being searched for.
- 🟡 **Middle value** — The value at the center of the current search area.
- ⬅️ **Left half** — The part before the middle.

✅ **Answer: B) Search the left half**

🟡 Q4 — MCQ

What does low represent in Binary Search?

- A) The target value
- B) The middle value
- C) The starting index of the search area
- D) The number of elements found

💡 **Hint:** Think about where the current search area begins.

📖 Glossary

- 🔵 **low** — Starting index of the current search area.
- 🔍 **Search area** — The portion of the array currently being checked.

✅ **Answer: C) The starting index of the search area**

🟢 Q5 — Fill in the Blank

In Binary Search, if:

target > numbers[mid]

we continue searching in the _____ **half**.



Hint: In a sorted ascending array, larger values are on which side?



Glossary

- 🎯 **target** — The value we want to find.
- 🟡 **mid** — The middle position of the search area.
- ➡️ **Right half** — The portion containing larger values in an ascending array.



Answer: right
